CS 647. HW 2 Xiyao Long

Overall:

This is an attempt to implement Raft system using Akka Finite State Machine. The implemented aspects inlude voting mechanism and log replication, not includes the execution in state machines or state machine replication. The program is written according to figure 2 in the Raft paper with most of the variable names adopting the names used in the figure as well.

The program have implemented “NodeData” class which stores data that are supposed to be stored on a server. The FSM actor implements the operation when it receives certain message in a certain state.

There is another non-FSM actor class “Client”, these actors act as client and sends “Raw messages ” to all the actors, while only FSM actors in the state “Leader ” has function “leaderReceiveClientMessage” implemented to handle it.

Run:

Use IntelliJ to run the program, the main function is in DiningHakkersOnFsm.java
The .java files are all in ./src/main/java/sample.fsm

Results:

The program have all the messages sending and receiving enabled.
The nodes has been observed in all three states.

Resources: Akka java documentations.
some documentation about java function such as manipulation about array list etc.

Bugs:

1. After running the program for some time the term of the node are usually much larger than the term of the message(therefore the term of the sending actor), this might be because in some places the term on the node is not updated by the message term properly.

2. Since HeartBeatRPC is supposed to be a special case of AppendEntry RPC,

according to the Raft paper, a server(any follower or leader) receiving this message should restart the timeout if their terms are the same. But if I do restart the timer, which I already commented out in the function “followerReceiveHeartBeatRPC” line 6&7, the program will get stuck in term 1.

3. Although the program is observed to execute function “handleAppendEntryRPC”, but I haven’t do Logging for how the server logs changes, therefore it’s not very obvious whether they have been replicated successfully. But all the functionality has been implemented including initializing and updating the nextIndex and matchIndex HashMap and updating the commitIndex.

4. Maybe better logging can be done to better check the program.

